ফার্মওয়্যার, Arduino এবং প্রথম LED

পাঠ 1 — 1IOT, SST-INF-01c — এমবেডেড ডিভাইস, ইনপুট ও আউটপুট, Arduino বোর্ড, পিন ও GND, LED ও রেজিস্টর, Tinkercad, প্রথম প্রোগ্রাম

03/11/2026

1 ফার্মওয়্যার করা মানে কী

আজ পর্যন্ত তোমরা এমন **প্রোগ্রাম** লিখেছ যা **কম্পিউটারে** চলে। সেখানে যা শেখো তা প্রতিটি অনুশীলনে কাজে লাগাও, আর ধীরে ধীরে অনুশীলনগুলো আসল প্রোগ্রামের মতো হয়ে ওঠে: এটি অ্যাপ্লিকেশন তৈরির অংশ, সোমবারের ক্লাস।

ফার্মওয়্যার (firmware) করা একটু আলাদা: প্রোগ্রামটি কম্পিউটারে চলে **না**, চলে একটি **এমবেডেড** **ডিভাইসে** (dispositivo embedded)।

সংজ্ঞা

এমবেডেড **ডিভাইস** এমন একটি যন্ত্র যা কম্পিউটারের মতো হিসাব করে, কিন্তু কম্পিউটারের সব কাজ করে **না**: এটি তৈরি হয় **একটি** **মাত্র** **কাজের** জন্য, বা খুব অল্প কয়েকটি কাজের জন্য। সাধারণত এটি একটি **ইলেকট্রনিক** **বোর্ড** (scheda elettronica)।

কম্পিউটারে তোমরা খেলতে পারো, ইন্টারনেট দেখতে পারো, Excel ব্যবহার করতে পারো, ভিডিও দেখতে পারো, সোশ্যাল ব্যবহার করতে পারো। **ফ্রিজ**, **ওভেন** বা **গাড়ির** **কন্ট্রোল** **ইউনিটের** ভেতরের কম্পিউটারও একটি কম্পিউটার, কিন্তু তাতে ভিডিও গেম খেলার কথা ভাবা যায় না।

1.1 বোর্ড এবং CPU

এমবেডেড ডিভাইস হলো একটি **ইলেকট্রনিক** **বোর্ড** যার ওপর একটি **CPU** বসানো আছে। **CPU** হলো **মস্তিষ্ক**: যে অংশটি হিসাব করে। একে **মাদারবোর্ডের** (scheda madre) সাথে গুলিয়ে ফেলো না, মাদারবোর্ড হলো পুরো বোর্ডটি। কম্পিউটারেও মাদারবোর্ড আর CPU থাকে; পার্থক্য হলো উদ্দেশ্যে: কম্পিউটার তৈরি হয় নানা কাজের জন্য, এমবেডেড বোর্ড একটি মাত্র কাজের জন্য।

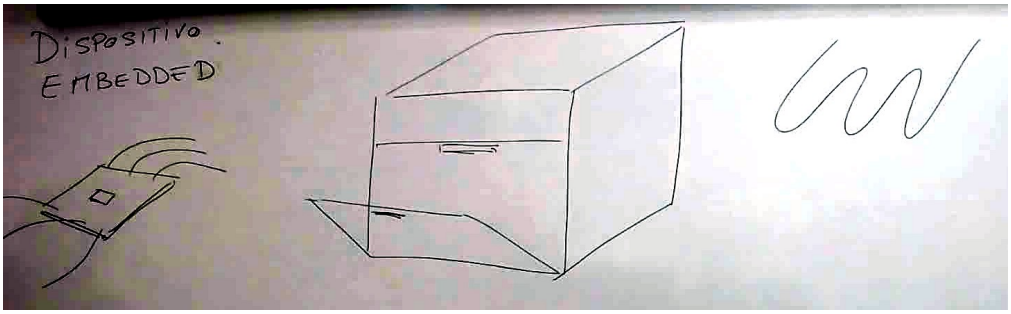
টীকা

বোর্ডগুলো প্রায়ই সবুজ কেন? বোর্ডের ওপর **রঙের** একটি স্তর থাকে, যা বৈদ্যুতিক পথগুলোকে **মরিচা** (ossidazione) থেকে রক্ষা করে। যেখানে যন্ত্রাংশ ঝালাই করতে হয়, শুধু সেখান থেকে এই রং তোলা হয়, **আলো** ব্যবহার করে এমন একটি পদ্ধতিতে (আলো-সংবেদী রং)। সবুজ হলো **ঐতিহাসিক** **রং**। এখন অন্য রঙের বোর্ড বানাতে খরচ কম, তাই নীল, লাল, সাদা, কালো, হলুদ, বেগুনিও পাওয়া যায়; তবে যেকোনো শেড নয়, কারণ রং অল্প কয়েকটিই পাওয়া যায়।

2 একটি উদাহরণ: বাড়ির ওভেন

ওভেন একটি **বাক্স**, যার একটি দরজা আর একটি হাতল আছে। ভেতরে, নিচে ও ওপরে, একটি **রেজিস্ট্যান্স** আছে, আকারের জন্য একে **সার্পেন্টিনা** (serpentina) বলে: এটি আমাদের ছোট রেজিস্টরগুলোর চেয়ে অনেক বড়। কারেন্ট

দিলে এটি **গরম** হয়, ওভেনের তাপমাত্রা বাড়ে আর খাবার রান্না হয়।



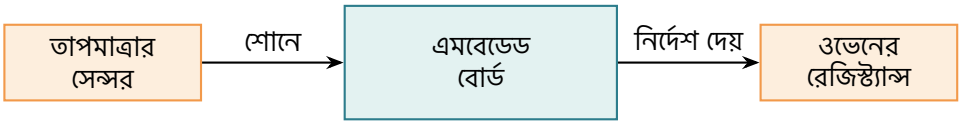
ছবি/নকশা 1: বোর্ডে আঁকা: তারসহ এমবেডেড বোর্ড, আর দরজা, হাতল ও সার্পেন্টিনাসহ ওভেন।

সমস্যা: রেজিস্ট্যান্স দশ ঘণ্টা চালু রাখলে তাপমাত্রা অনেক বেড়ে যায় আর খাবার **পুড়ে** যায়। তাই একটি **নিয়ন্ত্রণ** দরকার:

- রেজিস্ট্যান্স চালু হয় এবং গরম করে;
- ওভেনের ভেতরে একটি **সেন্সর** (sonda) তাপমাত্রা মাপে;
- **যদি** তাপমাত্রা 180 বা 200 ডিগ্রিতে পৌঁছায় (কী রান্না করছ তার ওপর নির্ভর করে), রেজিস্ট্যান্স **বন্ধ** হয়;
- **যদি** তাপমাত্রা সেই মানের নিচে নামে, আবার **চালু** হয়।

যদি শব্দটি লক্ষ করেছ? বোর্ডের প্রোগ্রামে সত্যিই একটি `if` থাকে। আর `printf`-এর বদলে থাকবে রেজিস্ট্যান্স চালু ও বন্ধ করার কমান্ড। এটি হলো তাপমাত্রার **স্বয়ংক্রিয় নিয়ন্ত্রণ**।

একই কাজ করা যায় **ফ্রিজে**, উল্টোভাবে, অথবা স্বয়ংক্রিয় **গেটের** কন্ট্রোল ইউনিটে: রিমোটের বোতাম চাপলে গেট খোলে, **যদি** মাঝখানে কেউ না থাকে। কথাগুলো প্রায় প্রোগ্রামের মতোই: বোর্ডকে এই ধরনের ঘটনা সামলানোর জন্যই প্রোগ্রাম করা হয়।



ছবি/নকশা 2: ওভেন: বোর্ড সেন্সরের কথা শোনে এবং রেজিস্ট্যান্সকে নির্দেশ দেয়।

3 ইনপুট ও আউটপুট: তার

কম্পিউটারে যোগাযোগ হয় **কীবোর্ড** আর **মনিটরে**। এমবেডেড বোর্ডে নয়: সেখানে যোগাযোগ হয় **বৈদ্যুতিক তারের** মাধ্যমে, যাতে **কারেন্ট দেওয়া বা বন্ধ করা** যায়। এটি ঠিক করে প্রোগ্রাম: কোনো তার ওভেনের রেজিস্ট্যান্সের সাথে যুক্ত থাকলে প্রোগ্রাম সেটি চালু বা বন্ধ করতে পারে। অন্য তারে সেন্সর যুক্ত থাকলে প্রোগ্রাম তার কথা **শুনতে** পারে।

তাই **দুই ধরনের তার** আছে: যেগুলো বোর্ডে **ঢোকে** আর যেগুলো **বের হয়**। সবই তার; বদলায় **তথ্যের দিক**।

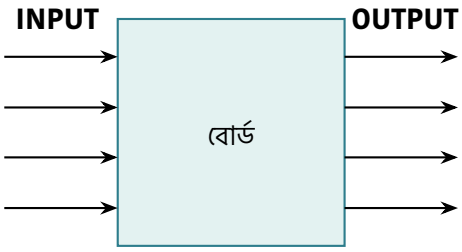
উদাহরণ

ভাবো তুমিই বোর্ড। **মুখ** দিয়ে তথ্য **বের হয়**: কী বলবে তুমি ঠিক করো। **কান** দিয়ে তথ্য **ঢোকে**: কান দিয়ে শব্দ তৈরি করা যায় না। বোর্ডেও তাই: কিছু তার দিয়ে বোর্ড বাইরের জগতের সাথে কথা বলে, অন্য তার দিয়ে বাইরের জগৎ কী বলছে তা শোনে।

সংজ্ঞা

যে তারগুলো বোর্ডে **ঢোকে** সেগুলোকে বলে **ইনপুট** (input): এখানে বাইরের জগৎ ঠিক করে, বোর্ড শোনে।
যে তারগুলো বোর্ড থেকে **বের হয়** সেগুলোকে বলে **আউটপুট** (output): এখানে বোর্ড ঠিক করে।

এই ছবিটি যেকোনো নথিতে পাবে: একটি বাক্স, আমাদের বোর্ড, ঢোকার তার **বাঁয়ে** আর বের হওয়ার তার **ডানে**, কারণ সাধারণত বাঁ থেকে ডানে পড়া হয়।



ছবি/নকশা 3: বোর্ডে আঁকা ছবি: বাঁয়ে ইনপুট, ডানে আউটপুট।

টীকা

প্রতিটি বিষয়ের জন্য একটি খাতা রাখো: একটি ইলেকট্রনিক্সের, একটি ফার্মওয়্যারের, অন্য বিষয়ের খাতায় ছড়ানো নোট নয়। একটিই ব্যবহার করতে চাইলে, উল্টো করে পেছন থেকে শুরু করো।

4 Arduino বোর্ড

অন্তত প্রথম বছর আমরা যে বোর্ড ব্যবহার করব তা হলো **Arduino**। মাঝখানে অনেক **ধাতব পা** সহ একটি কালো ছোট বাক্স আছে: এটি **চিপ**, আর প্রতিটি পা হলো বাইরের জগতের সাথে যোগাযোগের একটি তার। আসলে পুরো Arduino ওই চিপেই আছে; **সুবিধার** জন্য একে একটি বোর্ডে বসানো হয়, কারণ একা এটি ছোট এবং ধরা কঠিন।

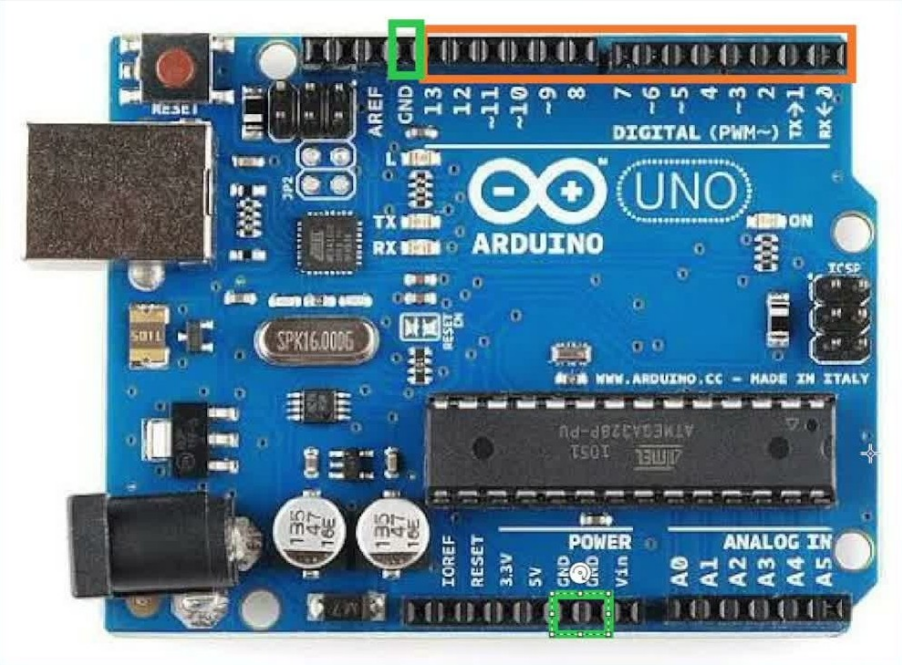
পাগুলো বোর্ডের ওপরে ও নিচের **কানেক্টরের** সাথে যুক্ত: ছোট ছোট **ছিদ্র**, যেখানে তার ঢোকালে আটকে থাকে। Arduino-র পাশে আমরা একটি সাদা প্লেট ব্যবহার করব, **breadboard**, যন্ত্রাংশ বসানোর জন্য, আর তার দিয়ে রেজিস্টর, বাতি, বোতাম যুক্ত করব: এভাবে দরকারি কাজ দিয়ে Arduino-কে **বাড়ানো** যায়।

4.1 কোথা থেকে এসেছে

Arduino-র জন্ম **2005** সালে, **ইতালিতে**, **Ivrea** শহরে, **Massimo Banzi**-র নেতৃত্বে একটি দলের হাতে। এটি শূন্য থেকে আবিষ্কার নয়: এ ধরনের বোর্ড আগেও ছিল। সমস্যা ছিল **শুরুর কঠিনতা**, যা ছিল অনেক বেশি:

- চিপের সাথে প্রায় 200 পৃষ্ঠার একটি **ম্যানুয়াল** আসত, প্রোগ্রাম করার আগে সেটি পড়তে হতো;
- YouTube ছিল না, ইন্টারনেটও আজকের মতো ছিল না: ম্যানুয়াল প্রায়ই বইয়ের দোকান থেকে কিনতে হতো, 80 ইউরো পর্যন্ত, 2 ইউরোর একটি চিপ প্রোগ্রাম করার জন্য;
- চিপকে একটি বাতি জ্বালাতে বলার আগে, শুধু সেটিংসের জন্যই লাগত প্রায় **200 লাইন কোড**।

ধারণাটি ছিল একটি বোর্ড বানানো যার ভেতরে আগে থেকেই একটি **মূল প্রোগ্রাম** আছে: বিরক্তিকর অংশটি তারা করে দিয়েছে, আর যে প্রোগ্রাম করে সে অল্প কয়েক লাইন লিখেই দরকারি জিনিস পায়। বাকিটা করেছে দাম: প্রায় **25 ইউরো**,



ছবি/নকশা 4: Arduino Uno। কমলা রঙে ডিজিটাল পিন, সবুজে GND পিন। কালো চিপ নিচে ডানে, USB কানেক্টর ওপরে বাঁয়ে।

যখন এমন একটি প্রোগ্রামিং ব্যবস্থার দাম হতে পারত 800-900 ইউরো। পরে এল চীনা কপি, 2-3 ইউরোয় বিক্রি।

সতর্কতা

কাজের জগৎ এভাবে চলে না। কোম্পানিতে যে চিপ ব্যবহার করতে চাও সেটি পড়ে শিখে চালাতে হয়, কারণ বিক্রির পণ্যের ভেতরে একটি Arduino থাকলে **খরচ অনেক বাড়ে**। আমরা এটি ব্যবহার করি **শেখার** জন্য, যন্ত্রের মতো ভাবতে আর বোর্ডের সাথে পরিচিত হতে। পরের বছরগুলোয়, ধাপে ধাপে, এই তৈরি ভিত্তি ছাড়া ব্যবস্থাগুলো দেখব।

Arduino-র মডেল আছে **অনেক**: আরও বড়, আরও পিনসহ, আরও শক্তিশালী চিপসহ, WiFi, Bluetooth, GPRS, LoRa ও অন্যান্য সংযোগসহ। আমরা ব্যবহার করি মূল মডেলটি, যাতে শুরুতে দরকারি অল্প কিছু জিনিস আছে। আগামী বছর WiFi-সহ একটি দেখব।

4.2 USB এবং ব্যাটারি

Arduino-তে একটি টাইপ B **USB** কানেক্টর আছে, বড় ও পুরোনো ধরনের; তারের অন্য প্রান্তে টাইপ A, যা কম্পিউটারে যায়। তারটি দুটি কাজ করে:

- বোর্ডকে **বিদ্যুৎ দেয়**, মানে চালু করে;
- বোর্ডের মেমরিতে **প্রোগ্রাম নামানোর** পথ খোলে।

চলার জন্য কম্পিউটারের সাথে যুক্ত থাকা **জরুরি নয়**: প্রোগ্রাম **বোর্ডের ভেতরেই চলে**। একবার প্রোগ্রাম করা হলে খুলে নিতে পারো, এমন মাঠে নিয়ে যেতে পারো যেখানে বিদ্যুৎ নেই, কানেক্টরে একটি **ব্যাটারি** লাগাও, আর এটি চলবে। ল্যাবে সুবিধার জন্য আমরা এটি USB-তে যুক্ত রাখব।

5 Arduino-র পিন

ওপরের কানেক্টরে **0** থেকে **13** নম্বরের ছিদ্র আছে, 7 আর 8-এর মাঝে একটি ফাঁকা জায়গা, একটি বিভাজন। সাবধান: শুরু 0 থেকে, তাই মোট **14**টি। এই ছিদ্রগুলোর কয়েকটি নাম: **পিন** (pin), **টার্মিনাল** (terminali), **কন্টাক্ট** (contatti), **পা** (piedini)। একই জিনিসের ভিন্ন নাম।

আগে আমরা ইনপুট ও আউটপুট আলাদা রেখেছিলাম। Arduino-তে কিন্তু 14টি পিন যেমন খুশি ব্যবহার করা যায়: কোনটি ইনপুট আর কোনটি আউটপুট, **আমরাই ঠিক করি**। 5 আউটপুট, 6 ইনপুট, 7 ইনপুট, 9 আউটপুট: প্রোগ্রামে ঠিক করা হয়।

সংজ্ঞা

Arduino-র **14টি ডিজিটাল পিন** আছে (0 থেকে 13), প্রতিটি **ইনপুট** বা **আউটপুট** হিসেবে ব্যবহার করা যায়।

5.1 ডিজিটাল মানে কী

ডিজিটাল মানে পিনের শুধু **দুটি অবস্থা**: চালু বা বন্ধ, মানে কারেন্ট দেয় বা দেয় না, আলোর সুইচের মতো, 1 বা 0-র মতো।

উদাহরণ

ফোন বা হেডফোনের **ভলিউম** কমানো-বাড়ানো যায়: শুধু সর্বোচ্চ বা বন্ধ নয়। এই পিনগুলো তা পারে না: হয় চালু, নয় বন্ধ। মাঝামাঝি, “একটু চালু কিন্তু বেশি নয়”, **হয় না**।

- বাতি জ্বলবে কি নিভবে আমি ঠিক করি: পিনটি **আউটপুট**।
- একটি সেন্সর আমাকে জানায় দরজা খোলা না বন্ধ, আর আমি শুনি: পিনটি **ইনপুট**।

5.2 GND: কারেন্ট যেখানে ফেরে

ধরো একটি 9 ভোল্টের **ব্যাটারি**, তার প্লাস ও মাইনাসসহ। **প্লাস** একটি বাতিতে যুক্ত করো, আর বাতিকে **মাইনাসে**: বাতি জ্বলে। তুমি একটি **সার্কিট** বানালে: কারেন্ট প্লাস থেকে বের হয়, বাতির মধ্য দিয়ে যায় আর মাইনাসে ফেরে।

Arduino-তেও একই। কারেন্ট একটি পিন থেকে (যেমন 7) বের হয়ে বাতির মধ্য দিয়ে গেলে তাকে কোথাও ফিরতে হবে। সেই জায়গা হলো **GND**, যা **ব্যাটারির মাইনাসের** সমান। GND মানে **ground**, অর্থাৎ “মাটি”। বোর্ডে তিনটি GND পিন আছে: একটি 13-এর পাশে, দুটি নিচের কানেক্টরে।

সতর্কতা

কোনো যন্ত্র **কখনো** একটি মাত্র **তারে** যুক্ত হয় না। অন্তত **দুটি** লাগে: একটি কারেন্ট নিয়ে যায়, অন্যটি ফিরিয়ে আনে, **GND**-তে। GND-র তার ছাড়া বাতি চলে না।

কেন একে মাটি বলে

বৈদ্যুতিক বিভব মাপা হয় **পৃথিবীর** বিভবের সাপেক্ষে, যা আমাদের **শূন্য** **রেফারেন্স**। ল্যাবের বৈদ্যুতিক প্যানেল খুললে নীল, কালো আর **হলুদ-সবুজ** তার দেখবে। হলুদ-সবুজটি হলো **আর্থ** **তার**, নিরাপত্তার জন্য: এটি মাটির নিচে যায়, প্রায় দুই মিটার গভীরে, একটি ধাতব প্লেটে, আর কারেন্ট মাটিতে ছড়িয়ে যায়, যেমন এক বালতি পানি ঢাললে মাটি শুষে নেয়। ওই তারকে **শূন্য** **ভোল্টের** রেফারেন্স ধরা হয়; তাই ব্যাটারি ও বোর্ডের মাইনাসকে **ground** বলে, এমনকি সত্যিকারের মাটিতে যুক্ত না থাকলেও। Arduino একে লেখে **GND**। এর একটি প্রতীকও আছে, কয়েকটি রূপে: বাঁয়েরটি সবচেয়ে পরিচিত, বিদ্যুতের খুঁটির প্লেটেও এটি পাবে।



ছবি/নকশা 5: মাটির প্রতীকগুলো (ground)।

⚠ কর্মক্ষেত্রে নিরাপত্তা

তোমরা নিজেরা **শূন্য** **ভোল্টে** আছ, কারণ মাটিতে দাঁড়িয়ে আছ। তাই **বাড়ির** **সকেট** বিপজ্জনক: আঙুল ঢোকালে কারেন্ট **শরীরের** **মধ্য** **দিয়ে** মাটিতে যায়, আর **মৃত্যু** হতে পারে। প্লাগ পয়েন্ট, বৈদ্যুতিক প্যানেল বা তার **কখনো** ছোঁবে না।

উদাহরণ

উচ্চ ভোল্টেজের তারে পাখি। বিদ্যুৎ সঞ্চালন লাইনে ভোল্টেজ অনেক বেশি। তাহলে পাখি মরে না কেন? কারণ তারা **একটি** **মাত্র** **তার** ছোঁয় আর মাটি ছোঁয় না: কারেন্টের যাওয়ার কোনো পথ নেই। কোনো মানুষ দুটি তার একসাথে, বা একটি তার আর মাটি ছুঁলে সাথে সাথে মারা যেত। তাই রক্ষণাবেক্ষণ খুব জটিল: কখনো একসাথে দুটি জায়গা না ছোঁয়া নিশ্চিত করতে হয়।

⚠ কর্মক্ষেত্রে নিরাপত্তা

উচ্চ ভোল্টেজের লাইনের কাছে **কখনো** যাবে না: তার **না** **ছুঁলেও** স্পার্ক হতে পারে।

টীকা

প্লাস ও মাইনাসের অদলবদল। বিদ্যুৎ নেটওয়ার্কে কারেন্ট **পরিবর্তী** (alternata): 50 Hz-এ একটি পূর্ণ চক্র 20 মিলিসেকেন্ড, আর প্লাস ও মাইনাস প্রতি 10 মিলিসেকেন্ডে **অদলবদল** হয়। এটি শক্তি পরিবহনের প্রযুক্তির সাথে জড়িত।

সঞ্চালন লাইনের কাছে। উচ্চ ভোল্টেজের লাইনের নিচে **বৈদ্যুতিক** **ক্ষেত্র** এত শক্তিশালী যে হাতে ধরা একটি **নিয়ন** **টিউব** জ্বলে ওঠে, কোনো তার ছাড়াই। আইন অনুযায়ী লাইনের চারপাশে একটি **নিরাপত্তা** **এলাকা** থাকে যেখানে বাড়ি বানানো যায় না: এর প্রস্থ লাইনের ওপর নির্ভর করে।

6 LED এবং রেজিস্টর

আমরা শুরু করব **বাতি** জ্বালিয়ে, কারণ এটি সবচেয়ে সহজ; পরে যোগ হবে বোতাম ও আরও জটিল যন্ত্র। আমাদের বাতির কারিগরি নাম **LED**: এমন একটি যন্ত্রাংশ যা কারেন্ট গেলে আলো দেয় এবং **খুব** **কম** **খরচ** **করে**।

আমাদেরগুলোতে একটি রঙিন বাল্ব (লাল, সবুজ বা হলুদ) আর **দুটি তার**, একটি **লম্বা** ও একটি **ছোট**; দৈর্ঘ্যের পার্থক্য সামান্য, ভালো করে দেখতে হয়।

সতর্কতা

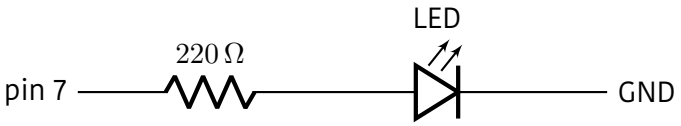
LED সরাসরি Arduino-তে যুক্ত করা যায় **না**: কারেন্ট এত বেশি হবে যে সেটি **পুড়ে** যাবে। সবসময় একটি **রেজিস্টর** দিয়ে যেতে হয়।

রেজিস্টরের দুটি টার্মিনাল আছে আর **কোনো দিক নেই**: যেকোনোভাবে লাগানো যায়। কিন্তু **LED-র দিক আছে**: উল্টো করলে কারেন্ট যায় না আর সার্কিট চলে না। কিছু না চললে আগে এটি পরীক্ষা করো।

সংজ্ঞা

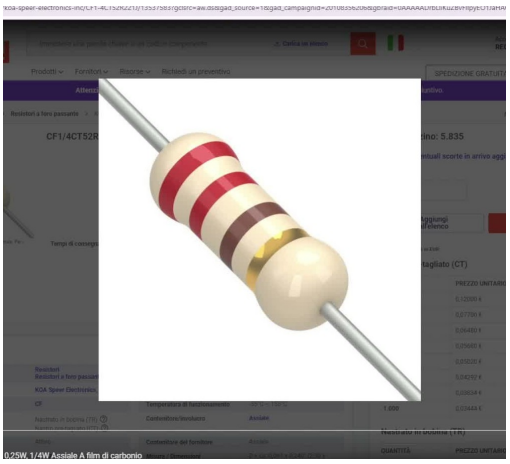
LED-র **লম্বা** তারকে বলে **অ্যানোড** (anodo): সেখান দিয়ে কারেন্ট **ঢোকে**।
ছোট তারকে বলে **ক্যাথোড** (catodo): সেখান দিয়ে কারেন্ট **বের হয়**।

নামগুলো সত্যিই কাজে লাগে: সিমুলেটরে LED-র তারের ওপর মাউস নিলে anodo বা catodo লেখা ওঠে।



ছবি/নকশা 6: সার্কিট: পিন, রেজিস্টর, LED (অ্যানোড রেজিস্টরের দিকে, ক্যাথোড GND-র দিকে)।

রঙিন ব্যান্ডগুলো রেজিস্টরের মান জানায়, যা মাপা হয় **ওহমে** (Ω)। কীভাবে পড়তে হয় তা ইলেকট্রনিক্সের ক্লাসে দেখবে। ওহম যত বেশি, রেজিস্টর তত বেশি কারেন্ট "আটকায়"। আমাদের পরীক্ষায় ব্যবহার করব **220 Ω** রেজিস্টর: **লাল, লাল, বাদামি**, আর শেষে একটি **সোনালি** ব্যান্ড। লিখে রাখো: আসল রেজিস্টর হাতে নিলে কাজে লাগবে।



ছবি/নকশা 7: রঙিন ব্যান্ডসহ একটি রেজিস্টর।

শুরুতে আমরা একটি **সিমুলেটর** ব্যবহার করব। একটু দক্ষ হলে, পরিকল্পনা হলো তোমাদের একটি **বাক্স** দেওয়া, যাতে থাকবে একটি Arduino, একটি breadboard, এক গোছা তার, 10টি LED আর কয়েকটি রেজিস্টর, সার্কিটগুলো হাতে বানানোর জন্য। ঝুঁকি একটিই: ফার্মওয়্যারের সকালে সেটি বাড়িতে ফেলে আসা। সেদিন এটি স্কুলে আনতে হবে।

7 সিমুলেটর: Tinkercad

7.1 Tinkercad-এ ঢোকা

1. দেখো ব্রাউজার তোমার **স্কুলের অ্যাকাউন্টে** যুক্ত আছে কি না। ওপরে ডানে, বন্ধ করার X-এর কাছে, **Verifica la tua identità** (তোমার পরিচয় যাচাই করো) বোতাম আসতে পারে: ক্লিক করে লগইন করো।
2. **Tinkercad** খোঁজো, K দিয়ে, আর সাইটে ঢোকো।
3. **Iscriviti** (নিবন্ধন, ওপরে ডানে) বেছে নাও, তারপর **Crea account personale** (শেষ বিকল্প) আর **Accedi con Google**।
4. দেশ হিসেবে **Italia** বেছে নাও, নইলে ঠিক পথে যাবে না, আর জন্মতারিখ দাও: অ্যাকাউন্টের জন্য বয়স অন্তত 14 বছর হতে হবে।
5. **Avanti** আর **Continua** দিয়ে এগিয়ে যাও।

7.2 সার্কিট আঁকা

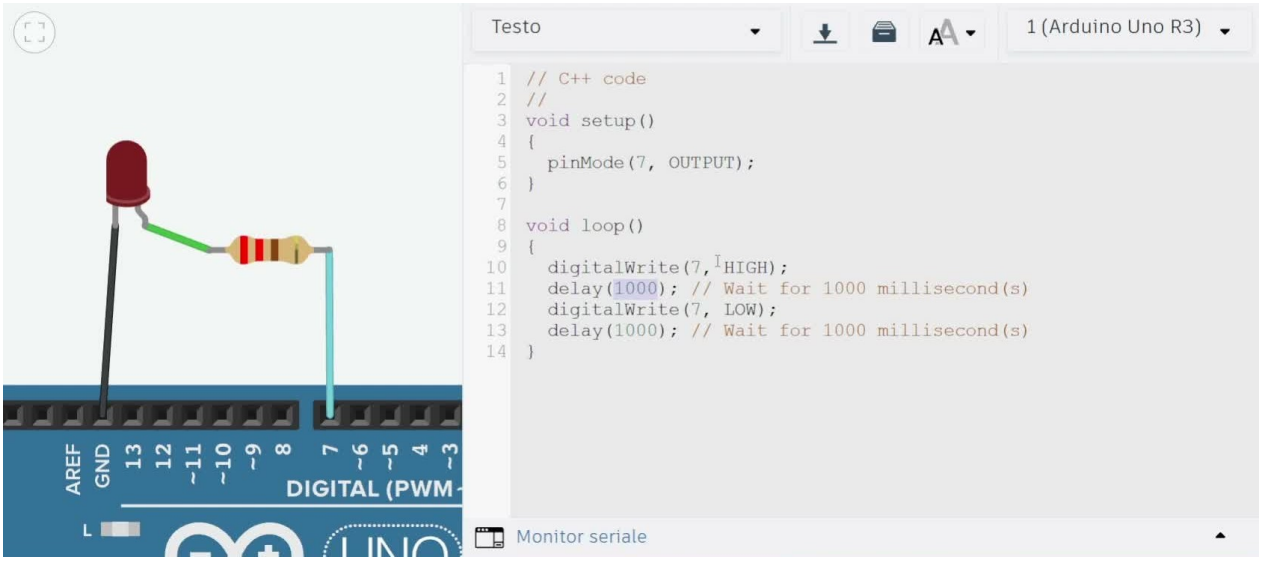
1. ওপরে ডানে **Crea** বেছে নাও, তারপর **Circuiti**। ডানে **যন্ত্রাংশের** তালিকা, সেগুলো টেনে পাতায় আনা হয়।
2. একটি **Resistenza** টেনে আনো। ক্লিক করলে চারপাশ নীল হয় আর ওপরে মান বদলানোর একটি ছোট মেনু আসে। শুরুতে এটি **kΩ**-এ থাকে (k মানে কিলো, অর্থাৎ হাজার): **220** লেখো আর **Ω** বেছে নাও। নাম গুরুত্বপূর্ণ নয়। হলুদ ব্যান্ড দেখলে কিছু ভুল আছে: হতে হবে লাল, লাল, বাদামি।
3. একটি **LED** টেনে আনো। ক্লিক করে রং বেছে নেওয়া যায়। LED-র রং দরকারি রেজিস্টরের মানকেও প্রভাবিত করে: নীল ও সাদায়, এই রেজিস্টর দিয়ে, নাও চলতে পারে। আপাতত **লাল** রাখো।
4. LED-র দুই পায়ের ওপর মাউস নিয়ে **anodo** ও **catodo** পড়ো।
5. তার জট না পাকাতে LED বাঁয়ে সরাও আর **R** বোতাম দিয়ে রেজিস্টর ঘোরাও।
6. তার টানতে: অ্যানোডে **ক্লিক**, তারপর রেজিস্টরের 1 নম্বর টার্মিনালে ক্লিক, বাঁয়েরটিতে। সংযোগ হয়ে গেল।
7. **Arduino Uno R3** টেনে আনো।
8. **ক্যাথোড GND**-তে যুক্ত করো। এক ক্লিকে তার বেছে নিয়ে ওপরের মেনু থেকে **কালো** রং নাও।
9. রেজিস্টরের অন্য টার্মিনাল **পিন 7**-এ যুক্ত করো, যে রঙে চাও।

টীকা

তারের রং। **GND**-র তার সবসময় **কালো**, **বিদ্যুৎ সরবরাহের** তার সবসময় **লাল**। সবুজ দিলেও সার্কিট চলবে, কিন্তু এটি একটি **প্রথা**: যে তোমার সার্কিট দেখবে, যেমন রক্ষণাবেক্ষণের জন্য, সে তারগুলো এভাবেই আশা করে। অন্য তারের রং তোমার পছন্দ।

সতর্কতা

সার্কিটে **কোনো ব্যাটারি নেই**: যোগ করো না। কারেন্ট দেয় Arduino। বাস্তবে ভুলভাবে যুক্ত ব্যাটারি **বোর্ড পুড়িয়ে** দেবে: Arduino স্কুল দেয়, কিন্তু পুড়িয়ে ফেললে আবার কিনতে হবে।



ছবি/নকশা 8: Tinkercad-এ প্রথম সার্কিট: পিন 7-এ LED ও রেজিস্টর, GND-তে কালো তার, সাথে তার কোড।

8 প্রথম প্রোগ্রাম

Codice বোতাম চাপো। Tinkercad **ব্লক** দেখায়, যা প্রাথমিক স্কুলের শিশুদের জন্য, কিন্তু আমরা আসল কোড লিখি: ওপরের মেনুতে Blocchi-র বদলে **Testo** বেছে নিয়ে **Continua** দিয়ে নিশ্চিত করি। একটি তৈরি প্রোগ্রাম আসে:

```
// C++ code
//
void setup()
{
  pinMode(LED_BUILTIN, OUTPUT);
}

void loop()
{
  digitalWrite(LED_BUILTIN, HIGH);
  // Wait for 1000 millisecond(s)
  delay(1000);
  digitalWrite(LED_BUILTIN, LOW);
  // Wait for 1000 millisecond(s)
  delay(1000);
}
```

`LED_BUILTIN` হলো বোর্ডে আগে থেকে বসানো LED। আমরা কিন্তু আমাদের LED পিন 7-এ যুক্ত করেছি, তাই যে **তিন** **জায়গায়** `LED_BUILTIN` লেখা আছে সেখানে **7** লিখি। অন্য পিন ব্যবহার করলে তার নম্বর লেখো। মন্তব্যের লাইনগুলো (`// C++ code`, `// Wait for...`) কাজে কোনো পরিবর্তন আনে না: নিচে পড়ার সুবিধার জন্য সরিয়ে দিয়েছি।

```
void setup()
{
  pinMode(7, OUTPUT);
}

void loop()
{
  digitalWrite(7, HIGH);
  delay(1000);
  digitalWrite(7, LOW);
  delay(1000);
}
```

Avvia simulazione চাপো: LED জ্বলে-নেভে। কোড বদলাতে আগে **Interrompi simulazione** চাপতে হয়: সবুজ বোতাম চালু থাকা পর্যন্ত কিছু বদলানো যায় না।

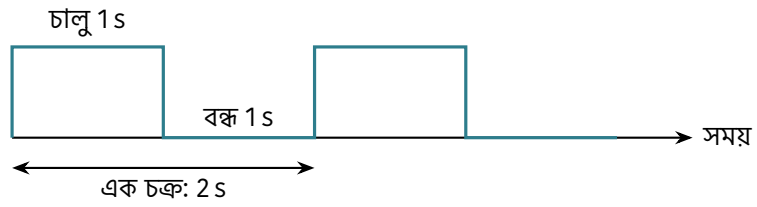
8.1 সময় নিয়ে খেলা

`delay`-এর ভেতরের সংখ্যা **মিলিসেকেন্ডে**: 1000 মিলিসেকেন্ড এক সেকেন্ড, 3 সেকেন্ড লেখা হয় **3000**, 10000 মানে 10 সেকেন্ড। দুটি মান বদলে দেখো:

- সংখ্যা যত **ছোট**, LED তত **দ্রুত** জ্বলে-নেভে;
- 60-90 মিলিসেকেন্ডের দিকে নামলে সেই সীমায় পৌঁছাও যেখানে চোখ জ্বলা-নেভা তখনও দেখতে পায়;
- 10 মিলিসেকেন্ডে এটি **সবসময়** **চালু** মনে হয়, কিন্তু তা নয়: 10 মিলিসেকেন্ড চালু, 10 বন্ধ। আমাদের **চোখ** তা দেখতে পারে না।

8.2 সময়ের চিত্র

এটি একটি **কার্তেসীয়** **চিত্র** (diagramma cartesiano)। এমন চিত্র অনেক দেখবে, তাই পড়তে শেখো। আনুভূমিক অক্ষে চলে **সময়**। **উঁচু** রেখা মানে LED **চালু**, **নিচু** রেখা মানে LED **বন্ধ**।



ছবি/নকশা 9: সময়ের সাথে প্রোগ্রাম: এক সেকেন্ড চালু, এক সেকেন্ড বন্ধ।

`loop`-এর ভেতরের কোড পড়ি:

- `digitalWrite(7, HIGH);` মানে “ডিজিটালে লেখো, পিন 7-এ, **উচ্চ** অবস্থা”: পিনে কারেন্ট দেয়, LED **জ্বলে**;
- `delay(1000);` যন্ত্র 1000 মিলিসেকেন্ড, মানে এক সেকেন্ড **থেমে** থাকে: `delay` ঘটনার **সময়** **ঠিক** করতে কাজে লাগে;
- `digitalWrite(7, LOW);` **নিম্ন** অবস্থা লেখে: LED **নেভে**;

- `delay(1000);` আরও এক সেকেন্ড অপেক্ষা;
- শেষ বন্ধনীতে পৌঁছে **আবার শুরু থেকে** চলে।

তাই একটি পূর্ণ চক্র **2 সেকেন্ড**।

টীকা

`digitalWrite` Arduino-র একটি কমান্ড, অনেকটা `printf`-এর মতো: `printf` ফল দেখায় মনিটরে, `digitalWrite` দেখায় LED-তে।

8.3 একটি মাত্র delay বদলানো: উজ্জ্বলতা

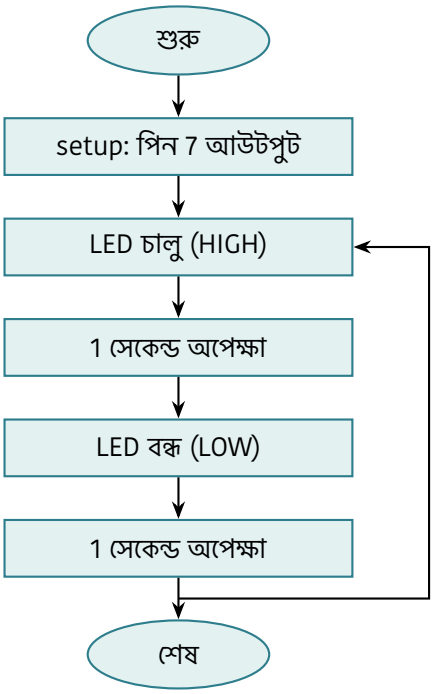
শুধু প্রথম delay কমালে LED অল্প চালু আর বেশি বন্ধ থাকে। এই কৌশল পরে দেখব, এটি **উজ্জ্বলতা নিয়ন্ত্রণে** কাজে লাগে। LED-র মাঝামাঝি অবস্থা নেই: সবসময় হয় চালু, নয় বন্ধ। 20-30 মিলিসেকেন্ডের নিচে নামলে জ্বলা-নেভা আর দেখা যায় না: 20 চালু আর 20 বন্ধে LED স্থির মনে হয়, শুধু একটু কম উজ্জ্বল। নিয়ন্ত্রণযোগ্য LED আলো এভাবেই চলে: এত দ্রুত **জ্বলে-নেভে** যে চোখ টের পায় না, আর চালু থাকার সময় বন্ধ থাকার সময়ের তুলনায় বদলে উজ্জ্বলতা বদলায়। সাধারণত LED চালু থাকার সময় কমালে বন্ধ থাকার সময় বাড়ানো হয়, যাতে চক্র সবসময় সমান থাকে।

8.4 loop এবং setup ব্লক

`loop`-এর কমান্ডগুলো দুটি **বাঁকা বন্ধনীর** (graffe) ভেতরে: এরা একটি নামসহ **ব্লক** তৈরি করে। এই বিষয়ের নাম **ফাংশন** (funzioni), পরে করব। আপাতত ভিডিও গেমের **ম্যাক্রোর** কথা ভাবো: ডান, বাঁ, নিচে, ওপরে, ঘুষি, লাথি চাপার বদলে পুরো ক্রমটি একটি বোতামে রেকর্ড করা হয়। ফাংশন অনেকটা এমনই: কমান্ডের একটি পাত্র, যেমন আমাদের `loop`-এর ভেতরের **4টি কমান্ড**।

সংজ্ঞা

`loop` হলো সেই ব্লক যা **অবিরাম চলতে থাকে**, যতক্ষণ বোর্ড চালু। দশ বছর চালু রাখলে কোটি কোটি বার চলবে।
`setup` হলো ওপরের ব্লক, একইভাবে তৈরি, যা চালু হওয়ার সময় **একবারই** চলে। আপাতত এতে একটিই কমান্ড: `pinMode`।
`pinMode` বলে দেয় পিনটি **ইনপুট** না **আউটপুট**। আপাতত সবসময় আউটপুট করি, `OUTPUT` দিয়ে।



ছবি/নকশা 10: জ্বলা-নেভা: `loop` সবসময় আবার শুরু হয়। বোর্ড আসলে কখনো শেষে পৌঁছায় না।

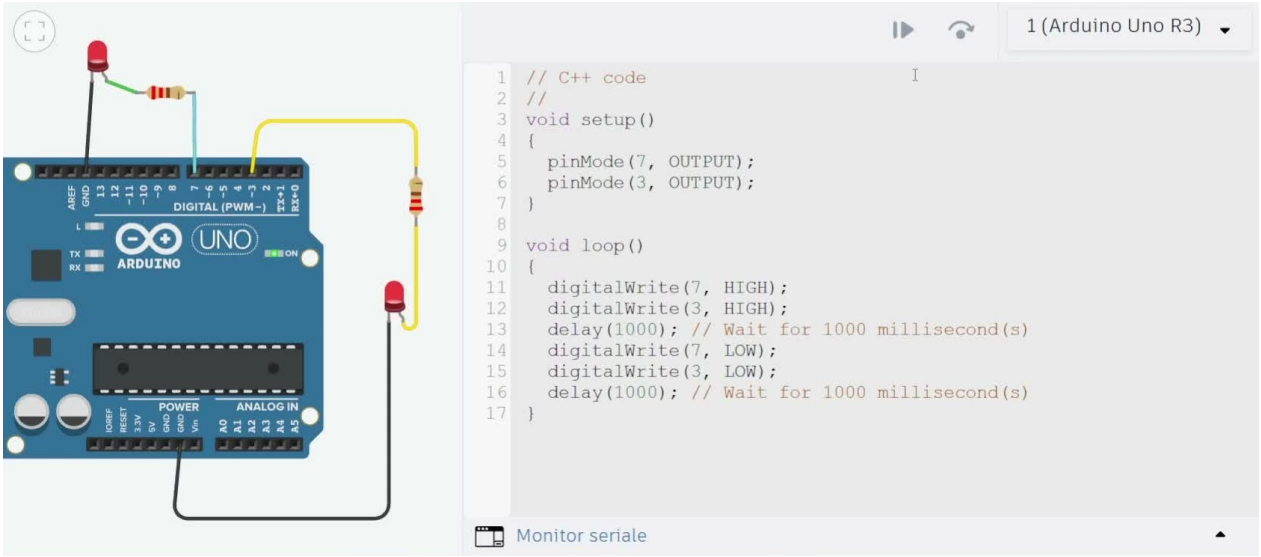
9 একসাথে দুটি LED

ক্লাসের অনুশীলন: অন্য একটি পিনে **দ্বিতীয়** **LED** যুক্ত করো, তার নিজের 220 Ω রেজিস্টরসহ, আর প্রথমটির সাথে একসাথে জ্বালাও-নেভাও। যা দরকার, দ্বিতীয় LED **পিন** **3**-এ ধরে:

1. `setup`-এ `pinMode(3, OUTPUT);` কারণ Arduino-কে জানতে হবে পিন 3-ও আউটপুট;
2. `loop`-এ `digitalWrite(7, HIGH);`-এর পরে যোগ করো `digitalWrite(3, HIGH);`
3. নিচে `digitalWrite(7, LOW);`-এর পরে যোগ করো `digitalWrite(3, LOW);`

```ino
void setup()
{
  pinMode(7, OUTPUT);
  pinMode(3, OUTPUT);
}

void loop()
{
  digitalWrite(7, HIGH);
  digitalWrite(3, HIGH);
  delay(1000);
  digitalWrite(7, LOW);
  digitalWrite(3, LOW);
  delay(1000);
}
```



ছবি/নকশা 11: পিন 7 ও 3-এ দুটি LED: ফেরার তার কালো এবং GND-তে যায়।

সতর্কতা

যে ভুলগুলো পরীক্ষা করবে:

- **আলাদা পিন**: প্রতিটি LED-র নিজের পিন। সিমুলেটরে একই পিনে দুটি LED চলছে মনে হয়, কিন্তু বাস্তবে এটি করা যাবে না, তাই শুরু থেকেই ঠিকভাবে অভ্যাস করো;
- **প্রতিটি পিনে একটি pinMode**: না থাকলে সেই LED ঠিকভাবে চলে না;
- **setup ও loop** প্রতিটি একটি করেই: সব `pinMode` একটি `setup`-এ, সব `digitalWrite` ও `delay` একটি `loop`-এ;
- **বড় হাতের অক্ষর**: `pinMode`-এ M বড়, `OUTPUT` পুরোটা বড়;
- **ফেরার** তার যায় **GND**-তে, সাধারণত কালো: যেকোনো GND চলে, দ্বিতীয় LED ফেরে নিচের কানেক্টরের GND-তে।

10 আগামী মঙ্গলবারের জন্য

উদাহরণ

Tinkercad-এ **দুটি LED**-র একটি সার্কিট বানাও যা **একবার এটি, একবার ওটি** জ্বলে: একসাথে নয়, আগে একটি, পরে অন্যটি। বাড়িতে **সিমুলেটর** ছাড়া আর কিছু লাগে না, এটি কাজ **নিজে থেকে সেভ** করে। **জমা দিতে হবে না**: পরের ক্লাসে দেখাবে। বিবরণ আছে **E01** শিটে।

11 এই পাঠ থেকে যা মনে রাখবে

- **ফার্মওয়্যার** চলে **এমবেডেড ডিভাইসে**: CPU-সহ একটি বোর্ড, একটি কাজের জন্য তৈরি।
- **ইনপুট** = যে তার ঢোকে, বোর্ড শোনে। **আউটপুট** = যে তার বের হয়, বোর্ড ঠিক করে।
- **Arduino**-র **14টি ডিজিটাল পিন**, 0 থেকে 13: শুধু চালু বা বন্ধ, ইনপুট বা আউটপুট ইচ্ছামতো।
- **GND** হলো মাইনাস: কারেন্টকে সবসময় ফিরতে পারতে হবে। অন্তত **দুটি তার** লাগে।
- **LED**-র দিক আছে (লম্বা **অ্যানোড**, ছোট **ক্যাথোড**) আর সবসময় একটি **রেজিস্টর** চায়: 220 Ω, লাল, লাল, বাদামি।
- তার: GND-র জন্য **কালো**, সরবরাহের জন্য **লাল**।
- `setup` চলে **একবার**, `loop` চলে **অসীমবার**।
- `pinMode(pin, OUTPUT)` পিন ঠিক করে, `digitalWrite(pin, HIGH)` বা `LOW` চালু বা বন্ধ করে, `delay(ms)` অপেক্ষা করে।

12 শব্দকোষ: ক্লাসে ব্যবহৃত ইতালীয় শব্দ

- **scheda**: বোর্ড

- **filo, fili**: তার
- **piedino, pin, terminale**: পা, পিন, টার্মিনাল
- **ingresso, uscita**: ইনপুট, আউটপুট
- **resistore, resistenza**: রেজিস্টর, রেজিস্ট্যান্স
- **anodo, catodo**: অ্যানোড, ক্যাথোড
- **forno, sonda**: ওভেন, সেন্সর
- **acceso, spento**: চালু, বন্ধ
- **accendere, spegnere**: জ্বালানো, নেভানো
- **lampadina**: বাতি
- **quaderno**: খাতা
- **graffe**: বাঁকা বন্ধনী
- **riga**: লাইন
- **Avvia simulazione, Interrompi simulazione**: সিমুলেশন শুরু, সিমুলেশন বন্ধ
- **Codice, Testo**: কোড, টেক্সট